31

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Saatler ve Zaman Sorunları: Saat Kayması (Clock Drift), NTP ve Vektör Saatleri

Distributed Clocks & Time: Physical Drift, NTP & Vector Clocks

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram

```
[ FİZİKSEL SAAT KAYMASI (CLOCK DRIFT & NTP SKEW) ]
Sunucu A Saati: [ 12:00:00.100 ] ──► Olay X (Gerçekte önce oldu!)
Sunucu B Saati: [ 12:00:00.050 ] ──► Olay Y (Saat geri kaldığı için önce gibi görünüyor!)
(Last-Write-Wins uygularsanız Olay X haksız yere ezilir ve silinir!)

[ MANTIKSAL VEKTÖR SAATI (VECTOR CLOCK) ]
Her düğüm kendi sayacını tutar: V = [NodeA: 1, NodeB: 0, NodeC: 2]
Eğer V1'deki tüm elemanlar <= V2 ise: V1 kesinlikle V2'den ÖNCE olmuştur (Happens-Before)!
Eğer çelişki varsa (V1 > V2 ve V2 > V1): Eşzamanlı çakışma (Concurrent Conflict) tespit edilir!
```

💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kolunuzdaki mekanik saatin her gün 2 saniye ileri gittiğini, arkadaşınızın saatinin ise 3 saniye geri kaldığını düşünün. İkiniz de 'Ben mesajı önce attım' diye kavga ederseniz kol saatlerinize bakarak kimin haklı olduğunu ASLA çözemezsiniz! Bilgisayarlardaki kuvars saatler de sıcaklıktan ve elektrikten dolayı kayar (Clock Drift). Dağıtık sistemlerde 'fiziksel duvardaki saate' güvenemezsiniz! Bunun yerine basketbol maçındaki paslaşma sırası gibi çalışan 'Mantıksal Saatler' (Lamport / Vector Clocks) kullanılır: 'Top önce bendeydi, ben sana attım, sonra sen kaleye attın; saatin kaç olduğu kimin umurunda!'

🇬🇧 ENGLISH (ELI5)

Imagine your wristwatch gains 2 seconds every day, while your friend's wristwatch loses 3 seconds. If you both send letters claiming 'I mailed mine first', comparing your wristwatches is completely useless! Computer quartz crystals drift due to thermal fluctuations (Clock Drift). In distributed systems, relying on physical wall-clock time (`System.currentTimeMillis()`) is dangerous! Instead, systems use Logical Clocks (Lamport & Vector Clocks): 'I held the ball first, passed it to you, and then you scored; physical clock ticks are irrelevant, causal sequence is everything!'

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerin en büyük yanılgısı: 'Fiziksel saatler eşzamanlıdır'. NTP (Network Time Protocol) saatleri senkronize etmeye çalışır ancak ağ dalgalanmaları nedeniyle sunucular arasında 10ms - 100ms arasında saat kayması (Clock Skew) kaçınılmazdır. 'Last-Write-Wins' (LWW) fiziksel saate dayanırsa veri kaybına yol açar (Google Spanner bunu çözmek için atomik saatler ve GPS alıcıları kullanan TrueTime API ile belirsizlik aralığını [ε ≈ 7ms] sınırlandırmıştır). Leslie Lamport'un Mantıksal Saatleri ve Vektör Saatleri (Vector Clocks), fiziksel saate ihtiyaç duymadan 'Happens-Before' nedensellik ilişkisini (A → B) matematiksel olarak modeller ve eşzamanlı çakışmaları (Concurrent Updates) tespit eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The fallacy of physical time in distributed nodes: NTP continuously steers hardware crystals, but asymmetric network jitter introduces 10ms-100ms clock skew. Naive Last-Write-Wins (LWW) relying on wall-clock time causes silent data loss. Google Spanner tackled this by placing GPS receivers and atomic clocks in every datacenter, bounding physical drift uncertainty to a narrow window (ε ≈ 7ms via TrueTime). Leslie Lamport proved that causality (A → B) can be modeled logically without physical clocks. Vector Clocks track an array of monotonic counters across nodes: if neither clock dominates the other, a concurrent conflict is flagged and handed to application logic for resolution.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Vektör Saati (Vector Clock) ile Nedensellik ve Çakışma Tespiti
import java.util.*;

public class VectorClock {
    private final Map<String, Integer> clock = new HashMap<>();

    public VectorClock() {}
    public VectorClock(Map<String, Integer> copy) { clock.putAll(copy); }

    public synchronized void tick(String nodeId) {
        clock.put(nodeId, clock.getOrDefault(nodeId, 0) + 1);
    }

    // İki vektör saatinin nedensellik durumunu karşılaştır
    public enum Relation { BEFORE, AFTER, CONCURRENT }

    public static Relation compare(VectorClock v1, VectorClock v2) {
        boolean v1Greater = false;
        boolean v2Greater = false;

        Set<String> allNodes = new HashSet<>(v1.clock.keySet());
        allNodes.addAll(v2.clock.keySet());

        for (String node : allNodes) {
            int c1 = v1.clock.getOrDefault(node, 0);
            int c2 = v2.clock.getOrDefault(node, 0);
            if (c1 > c2) v1Greater = true;
            if (c2 > c1) v2Greater = true;
        }

        if (v1Greater && !v2Greater) return Relation.AFTER;
        if (!v1Greater && v2Greater) return Relation.BEFORE;
        return Relation.CONCURRENT; // Çelişki / Eşzamanlı Çakışma!
    }
}

void main() {
    var v1 = new VectorClock();
    v1.tick("NodeA"); // [A:1]

    var v2 = new VectorClock(v1.clock);
    v2.tick("NodeB"); // [A:1, B:1] -> v2, v1'den sonra olmuştur!

    IO.println("V1 ve V2 İlişkisi: " + VectorClock.compare(v1, v2)); // BEFORE
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta dağıtık bir sistemde iki olayın hangisinin önce gerçekleştiğini belirlemek için `System.currentTimeMillis()` kullanacağınızı ASLA söylemeyin! NTP saati geriye atlayabilir (Leap Second veya NTP sync). Her zaman Mantıksal Saat veya Monotonic Sequence Number önerin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose sorting distributed transactions across multiple servers using `System.currentTimeMillis()`. NTP adjustments can cause system clocks to tick backwards! Always prescribe Logical Clocks, Lamport timestamps, or monotonic sequence counters.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fiziksel saatler donanım kayması ve NTP gecikmesi nedeniyle güvenilmezdir.

• Last-Write-Wins (LWW) saati geri kalan sunucudaki güncel verilerin silinmesine yol açabilir.

• Vektör saatleri (Vector Clocks) fiziksel zamana ihtiyaç duymadan çakışmaları tespit eder.

🎯 Key Takeaways for the Interview

• Physical clocks drift constantly due to hardware temperature and NTP transit jitter.

• Last-Write-Wins risks silently dropping valid writes if clock skew is present.

• Vector Clocks accurately trace causal order and isolate concurrent update conflicts.